

Hola Mundo Backend

Kotlin + Spring Boot + PostgreSQL

1. Tecnologías

Lenguaje	Framework	ORM	Base de datos
Kotlin 2.2.21	Spring Boot 4.0.6	Spring Data JPA (Hibernate)	PostgreSQL

2. Descargar el Java Development Kit (JDK)

Aunque el lenguaje del proyecto es Kotlin, instalar el [JDK](#) es obligatorio. Kotlin no puede ejecutarse por sí solo, ya que su compilador traduce el código a bytecode, un formato intermedio que solo la JVM sabe ejecutar. Spring Boot también corre sobre la JVM. Sin el JDK, ninguno de los dos funciona, sin importar que no se escriba Java en el proyecto.

En este caso, el proyecto usará **Java 21**, por lo que a continuación se muestra el proceso para descargar el **JDK 21**:

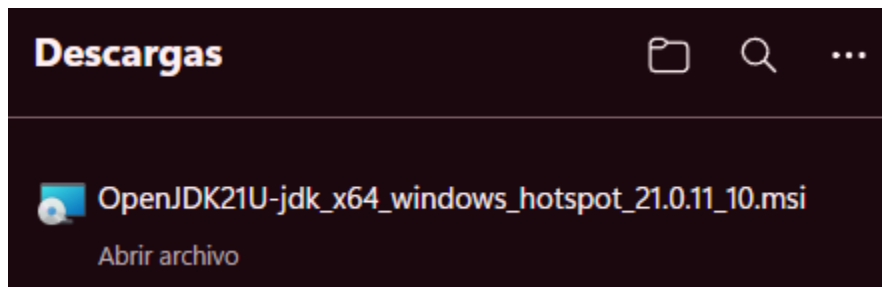
1. Ingresar a adoptium.net



2. Dar click en “otras descargas”
3. Seleccionar “JDK 21 - LTS” y dar click a descargar, dependiendo del sistema operativo en donde se va a instalar.



4. Abrir el instalador y seguir el proceso de instalación, dejando, por ahora, las opciones por defecto.



3. Configuración del Proyecto

1. Ingresar a start.spring.io
2. Configurar los parámetros del proyecto y agregar las dependencias.

The screenshot shows the Spring Initializr interface with the following configurations:

- Project:** ☐ Gradle - Groovy, ☒ Gradle - Kotlin, ☐ Maven
- Language:** ☐ Java, ☒ Kotlin, ☐ Groovy
- Spring Boot:** ☐ 4.1.0 (SNAPSHOT), ☐ 4.1.0 (RC1), ☐ 4.0.7 (SNAPSHOT), ☒ 4.0.6, ☐ 3.5.15 (SNAPSHOT), ☐ 3.5.14
- Project Metadata:**
 - Group:
 - Artifact:
 - Package name:
 - Packaging: ☒ Jar, ☐ War
 - Configuration: ☐ Properties, ☒ YAML
 - Java: ☐ 26, ☐ 25, ☒ 21, ☐ 17
- Dependencies:** (List of selected dependencies with descriptions and category tags like WEB, SECURITY, SQL, etc.)

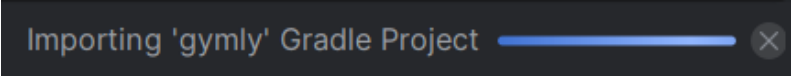
Buttons at the bottom: GENERATE (CTRL + G), EXPLORE (CTRL + SPACE), and a menu button (...).

Configuración	Valor
Project	Gradle - Kotlin
Language	Kotlin
Spring Boot	4.0.6
Group	app (Extensión de dominio)

Artifact	gymly (Nombre del proyecto)
Packaging	Jar
Configuration	YAML
Java	21

Dependencia	Uso
Spring Web	Para crear la api, crea endpoints y maneja peticiones y respuestas.
Spring Data JPA	Es el ORM, permite la interacción con la BD.
Validation	Verifica y valida los tipos de datos antes de procesarlos en la API.
Spring Reactive Web	Manejo de conexión masiva sin consumir tantos recursos.
Spring Boot DevTools	Kit de desarrollo que permite refrescar los cambios en el servidor sin resetearlo.
PostgreSQL Driver	Traduce los datos entre la API y BD.
Flyway Migration	Mantiene o controla las versiones de la BD.
Spring Boot Actuator	Monitorea el servicio de la aplicación, aportando métricas de estado del mismo.

3. Descargar el proyecto, pulsando el botón de “**Generate**”.
4. Descomprimir el zip y abrir la carpeta en un IDE (**preferiblemente IntelliJ IDEA**)
5. Esperar a que se importe las librerías y el proyecto de **Gradle**

A dark grey rectangular box with a blue progress bar and a close button (X) on the right. The text "Importing 'gymly' Gradle Project" is displayed in a light grey font.

4. Creación del ENV

```
DB_PORT=5432
DB_USER=<your_user>
DB_PASSWORD=<your_password>
DB_NAME=gymly_db
```

Es donde se configuran la base de datos a la cual está conectada el proyecto, en el usuario y contraseña se ponen las credenciales que se usan en el gestor de base de datos, en nuestro caso PostgreSQL

5. Levantamiento de la base de datos

1. Configurar archivo docker-compose.yml

```
services:
  db:
    image: postgres:15
    container_name: ${DB_NAME}
    restart: always
    ports:
      - "${DB_PORT}:5432"
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

El archivo contiene los servicios que se van a usar, para el proyecto se va a usar postgres en su versión 15, se usan las variables de nombre de la base, puerto, usuario y contraseña del .env y se crea el volumen de base de datos

2. Ejecutar el comando **docker-compose up -d** para levantar el contenedor con la base de datos
3. Configurar el archivo de creación de la base de datos con la definición de las tablas y relaciones

6. Configuración application.yml

1. Reemplaza el contenido de **src/main/resources/application.yml** para agregar la configuración de la base de datos y del ORM. Se debe suministrar un usuario y contraseña que pide la API para ingresar.

```
spring:
  application:
    name: gymly
  datasource:
    url: jdbc:postgresql://localhost:${DB_PORT}/${DB_NAME}
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

7. Definir la entidad

```
// src/main/kotlin/app/gymly/model/Role.kt

@Entity
@Table(name = "roles")
class Role(
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    var id: Int? = null,

    @Column(nullable = false)
    var name: String = ""
)
```

Anotación	Qué hace
@Entity	Le dice a JPA que esta clase representa una tabla en la base de datos
@Table(name = "users")	Especifica el nombre exacto de la tabla en PostgreSQL
@Id	Marca el campo como llave primaria de la tabla
@GeneratedValue(strategy = GenerationType.AUTO)	El valor del ID se genera automáticamente — AUTO deja que Hibernate elija la estrategia
@Column(nullable = false)	La columna no puede ser nula en la base de datos
@Column(name = "...")	Mapea el campo Kotlin a una columna con nombre distinto en la tabla

8. Crear el repository

Para crear el repositorio, se crea una interfaz que se extiende de la interfaz `JpaRepository`, la cual ya tiene suficientes métodos necesarios para que Spring se encargue de hacer las consultas a la base de datos sin necesidad de escribir sentencias en SQL manualmente. Se puede agregar la anotación `@Repository`, pero es opcional debido a que Spring Boot ya reconoce a `JpaRepository`.

```
// src/main/kotlin/app/gymly/repository/RoleRepository.kt

package app.gymly.repository

import app.gymly.model.Role
import org.springframework.data.jpa.repository.JpaRepository
import org.springframework.stereotype.Repository

@Repository
interface RoleRepository : JpaRepository<Role, Int>
```

9. Crear el service

```
// src/main/kotlin/app/gymly/service/RoleService.kt

package app.gymly.service

import app.gymly.model.Role
import app.gymly.repository.RoleRepository
import org.springframework.stereotype.Service

@Service
class RoleService(private val roleRepository: RoleRepository) {

    fun createRole(newRole: Role): Role {

        return roleRepository.save(newRole)

    }

    fun listRoles(): List<Role> = roleRepository.findAll()
```



```
}
```

10. Crear el controller

```
// src/main/kotlin/app/gymly/controller/RoleController.kt

package app.gymly.controller

import app.gymly.model.Role
import app.gymly.service.RoleService
import org.springframework.web.bind.annotation.*

@RestController
@RequestMapping("/")
class RoleController(private val roleService: RoleService) {

    @PostMapping("/rol")
    fun createRole(@RequestBody newRole: Role): Role {
        return roleService.createRole(newRole)
    }

    @GetMapping("/roles")
    fun listRoles(): List<Role> = roleService.listRoles()
}
```

11. Ejecutar la aplicación

Si el proyecto usa variables de entorno (archivo .env), el backend debe iniciarse ejecutando el script setup.sh desde un intérprete de bash como Git Bash.

Cuando se compila y levanta el proyecto correctamente, Spring lo confirmara en la terminal/consola.

Para correr el proyecto desde IntelliJ o cualquier IDE se realiza la siguiente configuración en el build.gradle.kts

```
// build.gradle.kts
```

```

tasks.named<org.springframework.boot.gradle.tasks.run.BootRun>("bootRun") {
    val envFile = file(".env")
    if (envFile.exists()) {
        envFile.readLines().forEach { line ->
            if (line.isNotBlank() && !line.startsWith("#")) {
                val parts = line.split("=", limit = 2)
                if (parts.size == 2) {
                    environment(parts[0].trim(), parts[1].trim())
                }
            }
        }
    }
}

```

Se ejecuta el comando **./gradlew bootRun** desde la terminal y springboot configura el proyecto para que corra el proyecto localmente.

```

PS C:\Users\sebas\Documents\WASD-UNAL\project-backend> ./gradlew bootRun
Starting a Gradle Daemon (subsequent builds will be faster)

> Task :bootRun

  .   ____          _            __ _ _
/\ \  / ___\      / \   __   \ / \ / \
(  )  \___ \    /   \_|| |  / \ V / \
\ \ / ____) |  /    \ | | /   \| | | |
 ' / |___) | /  ___/ | | | |___| | | |
=====|_|=====|___/___/___/___/

:: Spring Boot ::                (v4.0.6)

```

```

[#####...] 85% EXECUTING [1m 20s]
> :bootRun

```

12. Prueba desde Postman

1. Crear un registro Role en la base de datos

Request en Postman:

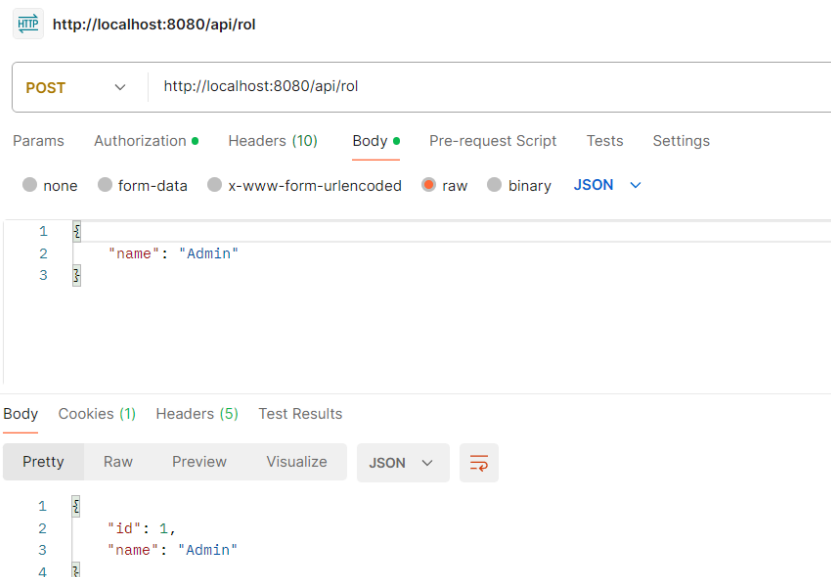
Campo	Valor
Método	POST
URL	http://localhost:8080/api/rol
Body type	raw → JSON

Body JSON:

```
{
  "name": "Admin"
}
```

Respuesta:

```
{
  "id": 1,
  "name": "Admin"
}
```



2. Obtener los Roles (GET)

Campo	Valor
Método	GET
URL	http://localhost:8080/api/roles

Respuesta:

```
[  
  {  
    "id": 1,  
    "name": "Admin"  
  }  
]
```